#초급반 4주차

그래프 이론, DFS, BFS

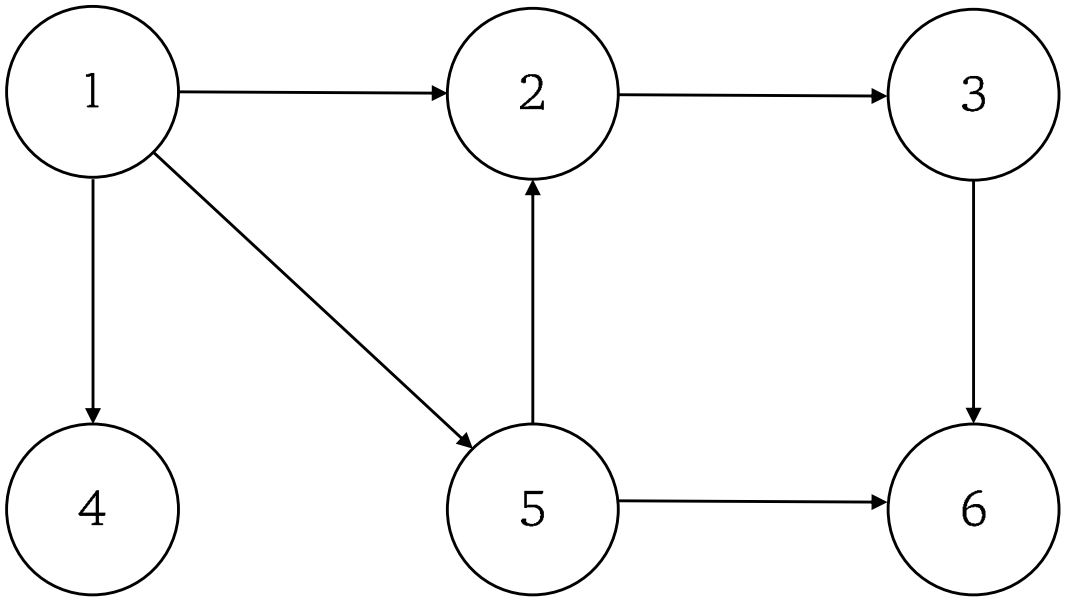
<Graph Theory, DFS, BFS>

2p /<그래프 이론>
11p /<DFS>
26p /<BFS>
44p /<연습 문제>

#그래프 이론
<Graph Theory>

그래프란?

정점(Vertex)과 간선(Edge)으로 이루어진 구조입니다.



#그래프 이론

<Graph Theory>

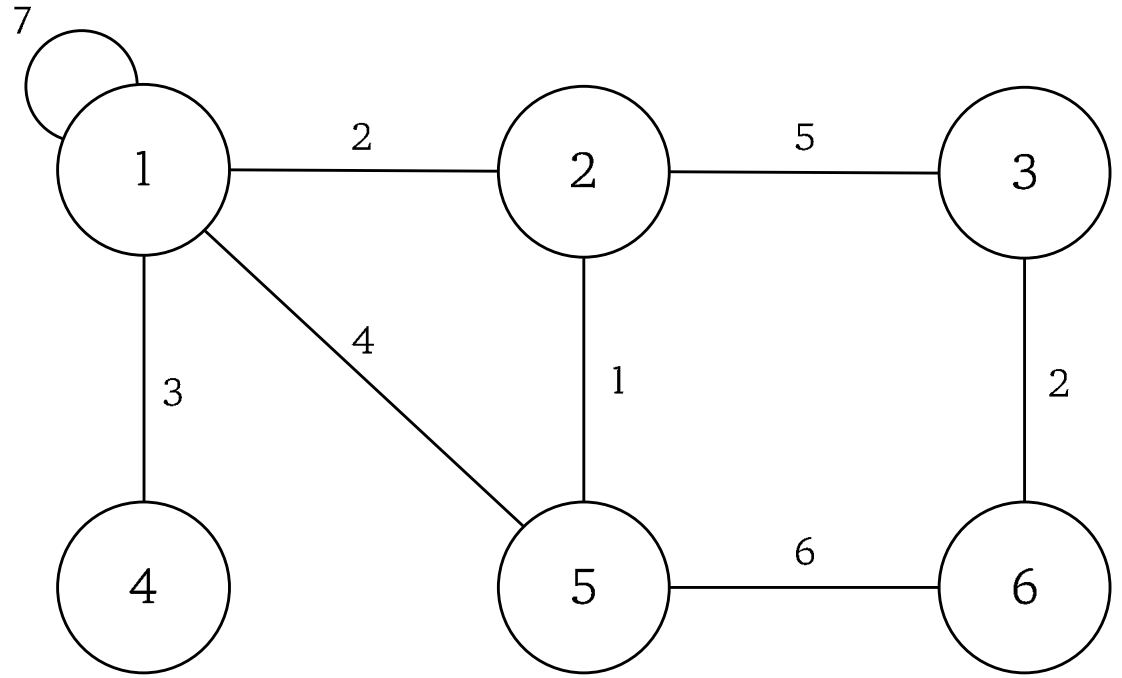
정점(노드) : 그래프를 이루는 객체(꼭짓점)
vertices(nodes)

간선 : 정점을 연결하는 선(변)
edges

루프 : 시작과 끝이 같은 간선
loop

가중치 : 그래프의 간선에 붙는 수치
weight

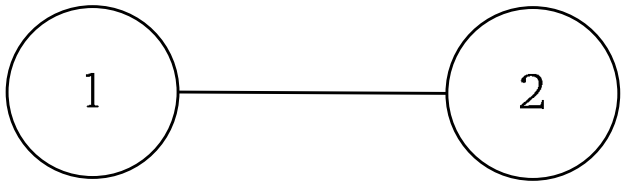
그래프 : 정점과 간선으로 이루어진 구조
graphs



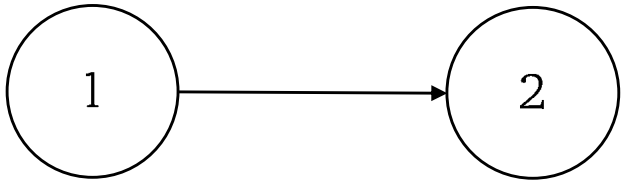
#그래프 이론

<Graph Theory>

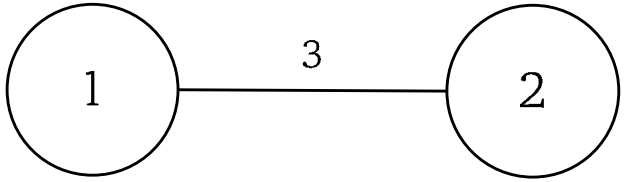
양방향 그래프 : 간선의 방향이 없는 그래프
bidirected graph(undirected graph)



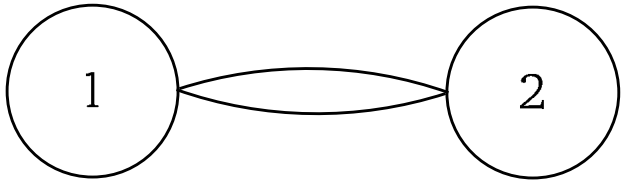
단방향 그래프 : 간선의 방향이 있는 그래프
directed graph(digraph)



가중치 그래프 : 간선에 가중치가 붙은 그래프
weighted graph



다중 그래프 : 두 정점을 잇는 간선이 여러 개일 수 있는 그래프
multigraph



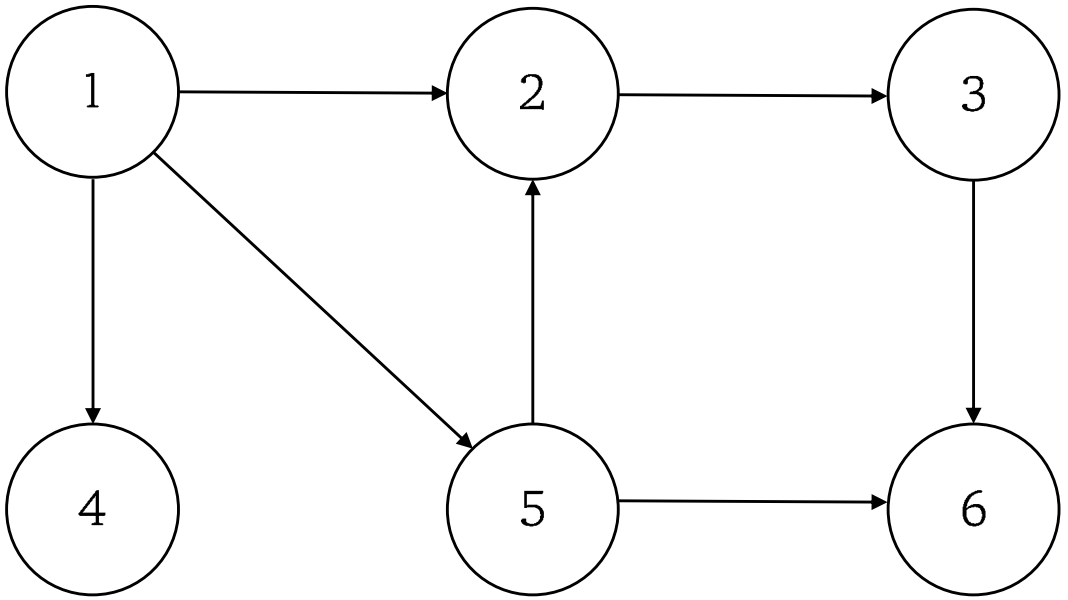
#그래프 이론
<Graph Theory>

그래프를 저장하는 방법은
다음과 같습니다.

1. 인접 행렬: 가능한 모든 간선의 연결여부 저장

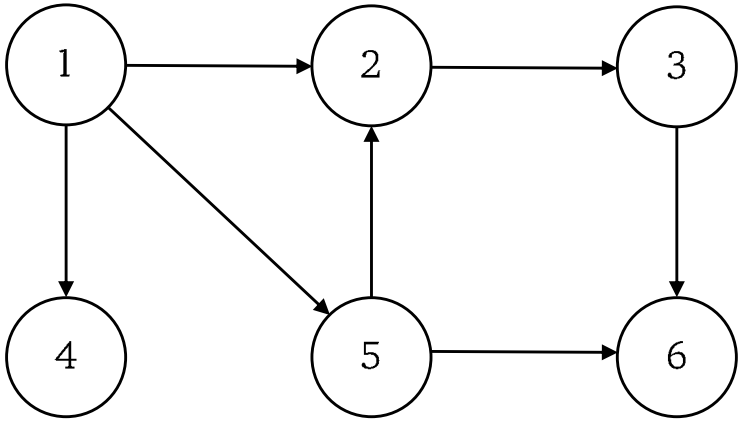
2. 간선 리스트: 그래프를 이루는 간선을 저장

3. 인접 리스트: 정점에서 연결된 모든 정점을 저장



#그래프 이론

<Graph Theory>



인접 행렬: 가능한 모든 간선의 연결여부 저장

정점 X-> 정점 Y로 가는 간선이 있을 때 G[X][Y]=1, 없다면 0의 형식으로 저장합니다.
총 V개의 정점이 있다면 V*V의 2차원 배열로 저장합니다.

가중치가 있는 경우 G[X][Y]=w 와 같은 방법으로 저장하게 됩니다.

$$G = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

#그래프 이론

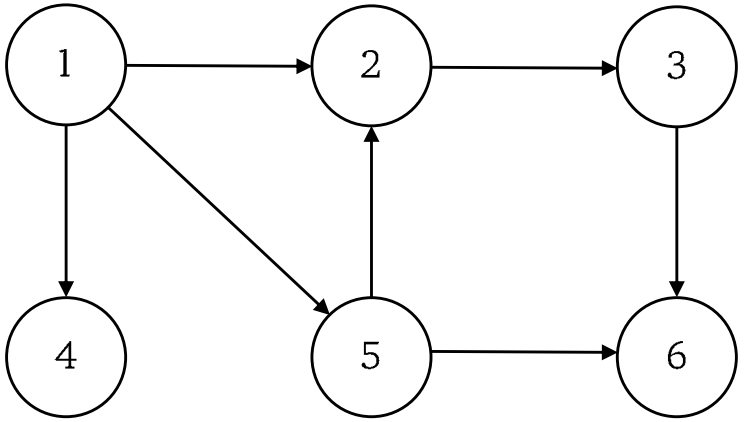
<Graph Theory>

간선 리스트: 그래프를 이루는 간선을 저장

정점 X-> 정점 Y로 가는 간선을 [X, Y]의 형태로 리스트에 모두 저장하는 방법입니다.

총 E개의 간선이 있으면 [X, Y]가 E개 있는 배열의 형태로 저장됩니다.

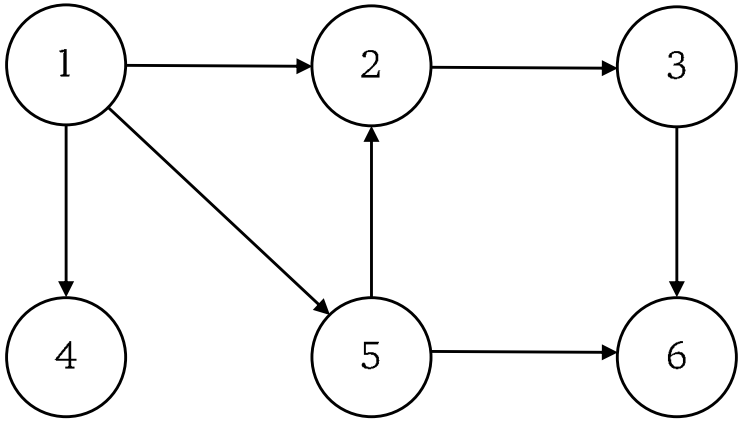
가중치가 있는 경우, [X, Y, w]의 형태로 저장하게 됩니다.



$$G = [[1,2],[1,4],[1,5],[2,3],[3,6],[5,2],[5,6]]$$

#그래프 이론

<Graph Theory>



인접 리스트: 정점에서 연결된 모든 정점을 저장

정점X에서 갈 수 있는 모든 정점을 G[X]에 리스트 형식으로 저장하는 방법입니다.

V개의 리스트가 들어있는 2차원 리스트의 형식으로
총 E개의 원소가 저장되게 됩니다.

가중치가 있으면 [Y, w]를 G[X]에 대신 저장하게 됩니다.

일반적으로 가장 많이 사용하는 그래프 저장방법입니다.

G[1] = [2, 4, 5]
G[2] = [3]
G[3] = [6]
G[4] = []
G[5] = [2, 6]
G[6] = []

#그래프 이론
<Graph Theory>

인접 리스트: 정점에서 연결된 모든 정점을 저장

인접 리스트는 다음과 같이 구현할 수 있습니다.

문제의 제한조건에 따라서 적절히 그래프를 저장하는 vector 배열을 선언 후, 간선을 추가하여 그래프를 저장하게 됩니다.

```
vector<int> g[1234];

int main(){
    cin.tie(0); cout.tie(0);
    ios::sync_with_stdio(0);
    int v, e;
    cin >> v >> e;
    for(int i = 0; i < e; i++){
        int x, y;
        cin >> x >> y;
        g[x].push_back(y);
      //g[y].push_back(x); 양방향 그래프인 경우 추가
    }
}
```

#그래프 이론

<Graph Theory>

그래프를 저장하는 세 가지 방법을 알아보았습니다.

1. 인접 행렬: 가능한 모든 간선의 연결여부 저장
   임의의 두 정점이 연결되어 있는지 빠르게 확인이 가능합니다.
   한 정점에서 연결되어 있는 정점을 확인하는 것이 느립니다.

2. 간선 리스트: 그래프를 이루는 간선을 저장
   간선을 기준으로 처리하는 경우, 효율적으로 처리가 가능합니다.
   대부분의 경우 다른 두 저장 방법을 사용합니다.

3. 인접 리스트: 정점에서 연결된 모든 정점을 저장
   한 정점에서 연결되어 있는 정점을 빠르게 확인할 수 있습니다.
   임의의 두 정점의 연결 여부를 확인하는 것이 느립니다.

이처럼 장단점이 다르기에 사용하는 알고리즘에 따라서
적절히 선택하면 됩니다.

저장 방식	사용 알고리즘
인접 행렬	플로이드-워셜, 인접행렬 거듭 제곱 관련 문제
간선 리스트	밸만포드, MST
인접 리스트	DFS, BFS, 다익스트라, MST 등

#DFS

<Depth First Search>

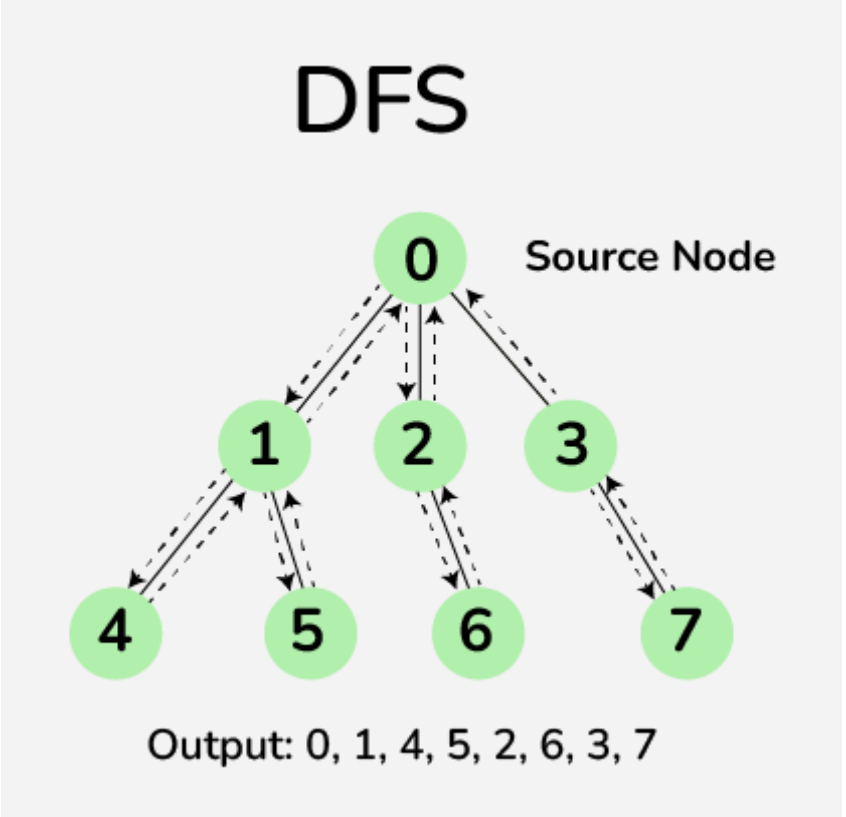
DFS:Depth First Search(깊이 우선 탐색)

그래프를 순회하는 방법 중 하나입니다.

현재 탐색하는 경로를 끝까지 탐색하는 것을
반복하는 방식으로 그래프에 있는 모든 정점을 순회합니다.

구체적으로는 아래의 과정을 반복합니다.

1. 방문하지 않은 정점 중 하나를 선택한다.
2. 현재 정점에서 갈 수 있는 방문하지 않은 정점을 방문한다.
3. 만약 2에서 더 이상 방문할 수 없으면 이전 정점으로
   돌아가여 반복한다.
4. 방문하지 않은 정점이 남아 있다면 1을 반복



https://www.geeksforgeeks.org/difference-between-bfs-and-dfs/

#DFS

<Depth First Search>



바이러스 (JUNGOL #1818)

<문제 설명>

- 컴퓨터가 n개 있으며 네트워크 상 연결 m개가 존재한다.
- 1번 컴퓨터가 바이러스에 감염이 되었다.
- 네트워크 상 연결을 통하여 바이러스가 퍼져간다.
- 1번 컴퓨터를 통하여 감염되는 컴퓨터의 수를 출력한다.

<제약조건>

$1 \le n \le 100$

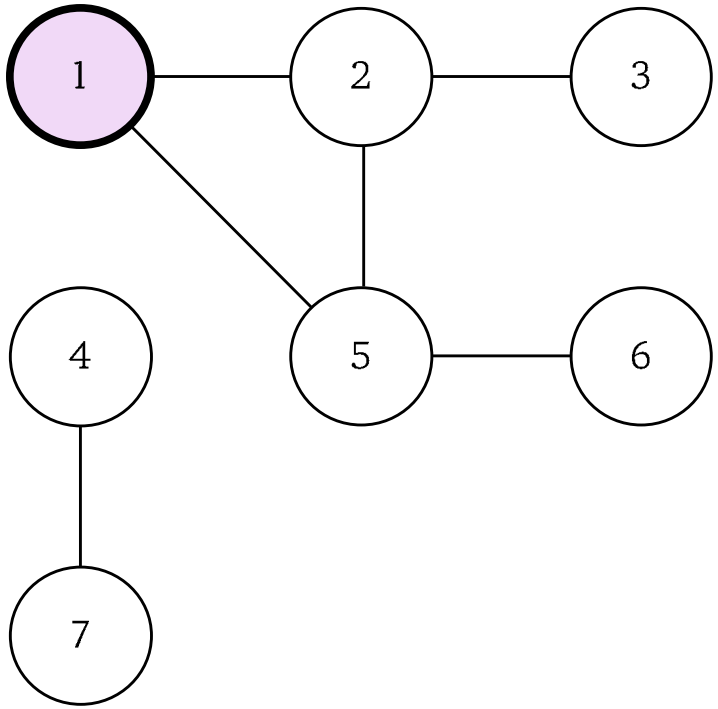
#DFS

<Depth First Search>

예제 그래프에서 DFS의 과정을 알아보겠습니다.

테두리가 두꺼운 정점은 현재 탐색하는 정점,
연보라색으로 칠해진 정점은 방문 된 정점,
보라색으로 칠해진 정점은 방문이 완료된 정점,
흰색은 방문하지 않은 정점입니다.

우선 방문하지 않은 정점 중 하나를 골라 탐색을 시작합니다. 지금의 경우 1번 정점을 선택하여 시작하게 됩니다.

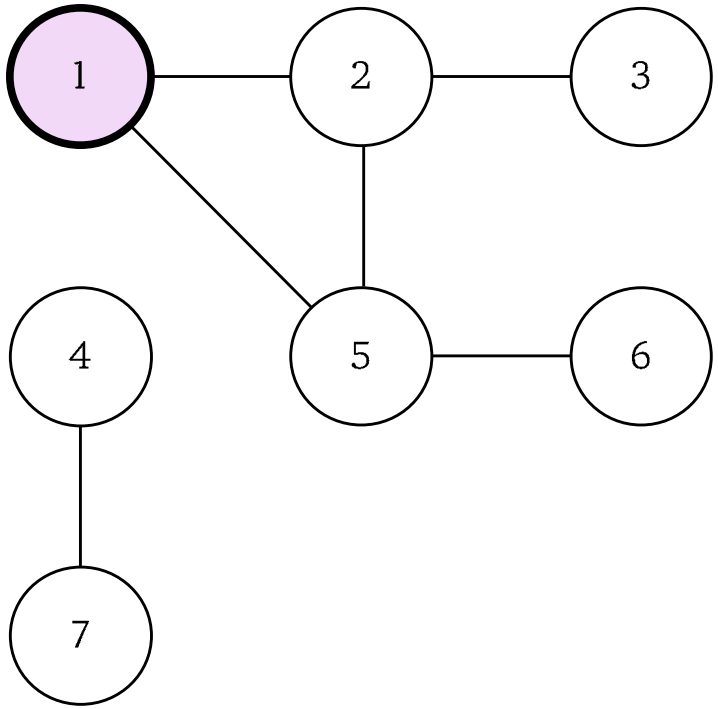


방문한 순서: 1
방문완료한 순서:

#DFS

<Depth First Search>

1번 정점에서 갈 수 있는 정점은 2번, 5번 정점이 있습니다. 이처럼 한 정점에서 갈 수 있는 정점이 여러 개가 있다면 아무 정점이나 골라도 됩니다.
여기서 5번 정점을 방문하겠습니다.



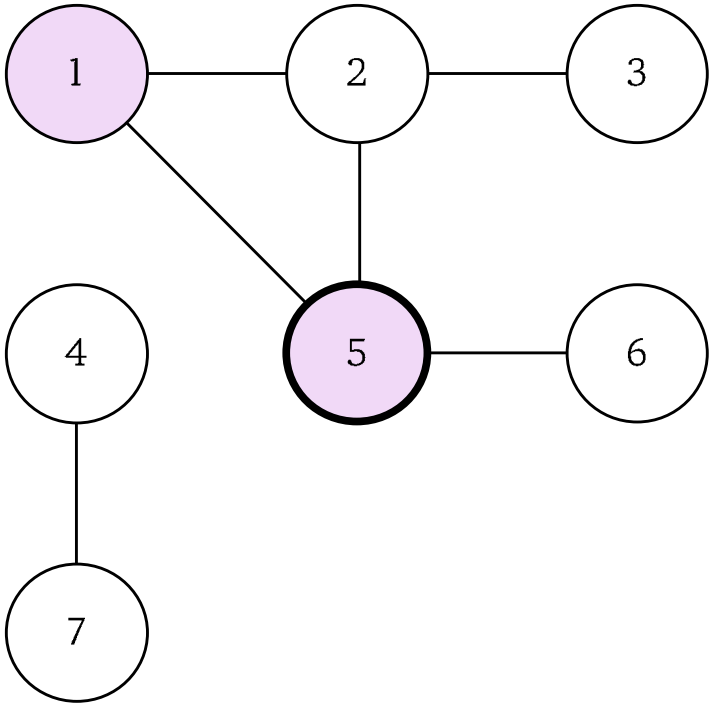
방문한 순서: 1
방문완료한 순서:

#DFS

<Depth First Search>

5번 정점에서 갈 수 있는 정점은 1번, 2번, 6번 정점이 있습니다. 1번 정점은 이미 방문했기에 2번, 6번 중 6번 정점을 다음에 방문합니다.

이처럼 이미 방문한 정점은 다시 방문하지 않습니다.

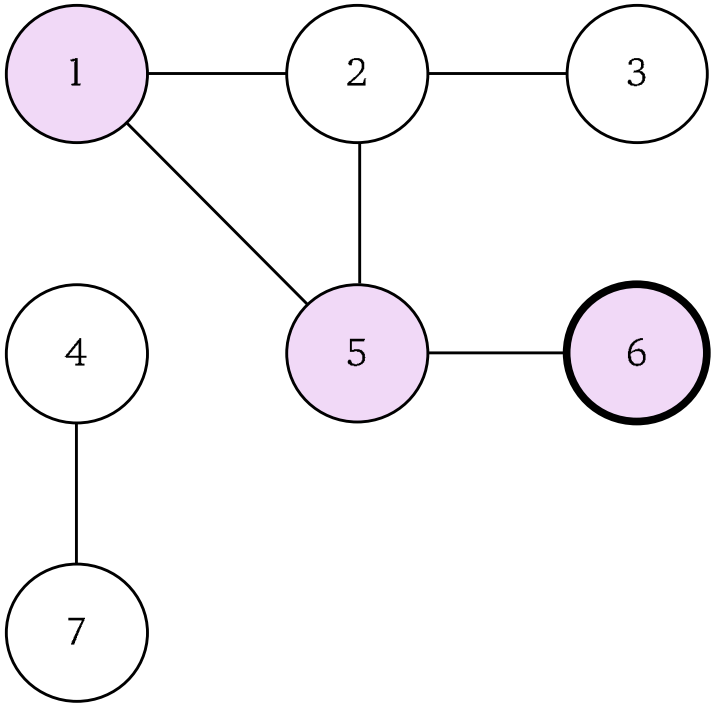


방문한 순서: 1, 5
방문완료한 순서:

#DFS

<Depth First Search>

6번 정점에서 갈 수 있는 방문하지 않은 점은 없습니다.
따라서 6번 정점은 방문이 완료되고 이전 정점인 5번 정점으로 되돌아갑니다.

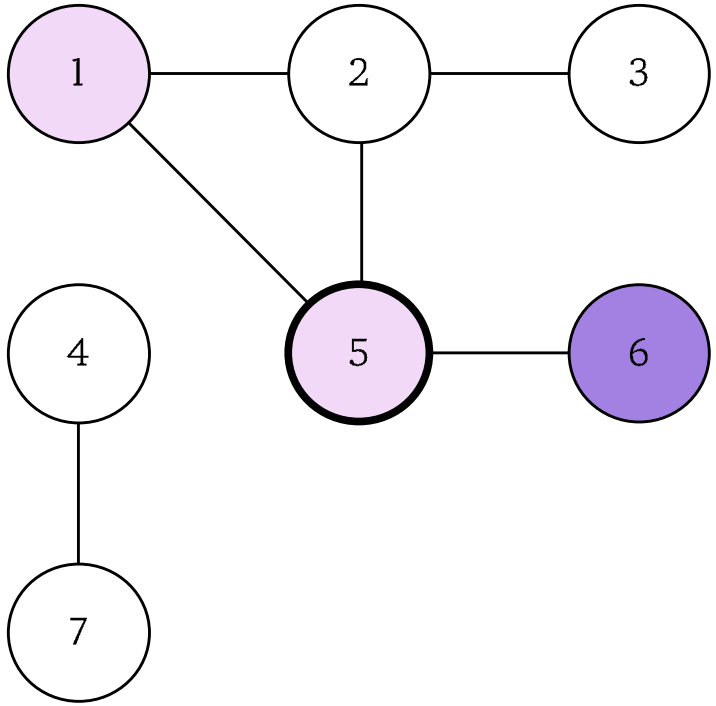


방문한 순서: 1, 5, 6
방문완료한 순서: 6

#DFS

<Depth First Search>

5번 정점에서 방문할 수 있는 점은 2번 정점이 남아있습니다.
따라서 2번 정점을 방문하게 됩니다.

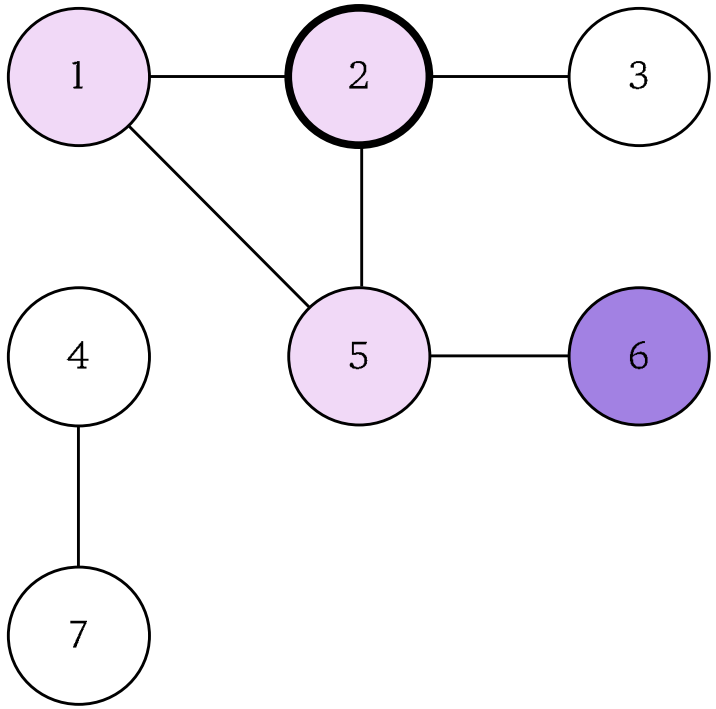


방문한 순서: 1, 5, 6
방문완료한 순서: 6

#DFS

<Depth First Search>

2번 정점에서 방문할 수 있는 정점은 3번 정점이 있습니다.
따라서 3번 정점을 방문하게 됩니다.



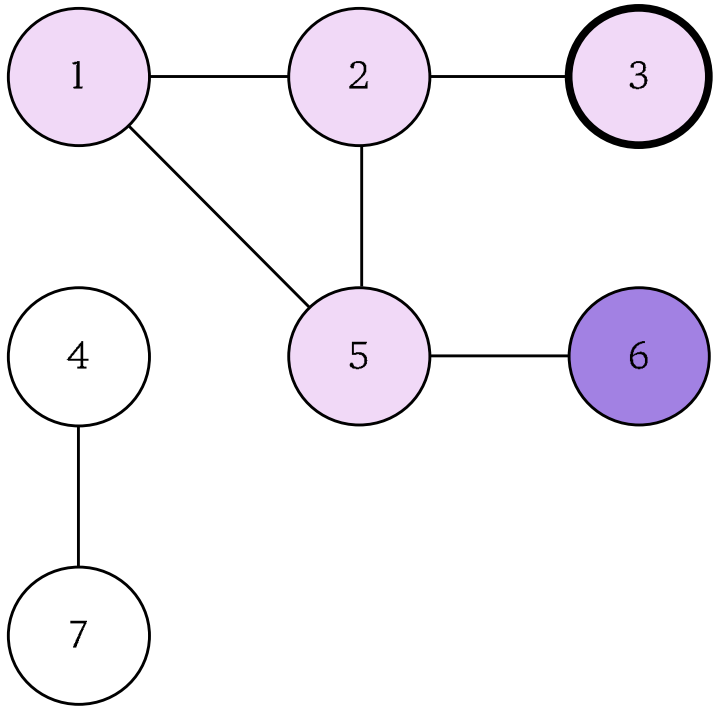
방문한 순서: 1, 5, 6, 2
방문완료한 순서: 6

#DFS

<Depth First Search>

3번 정점에서 방문 가능한 점은 없습니다.

방문을 완료하고 이전 정점인 2번 정점으로 되돌아갑니다.



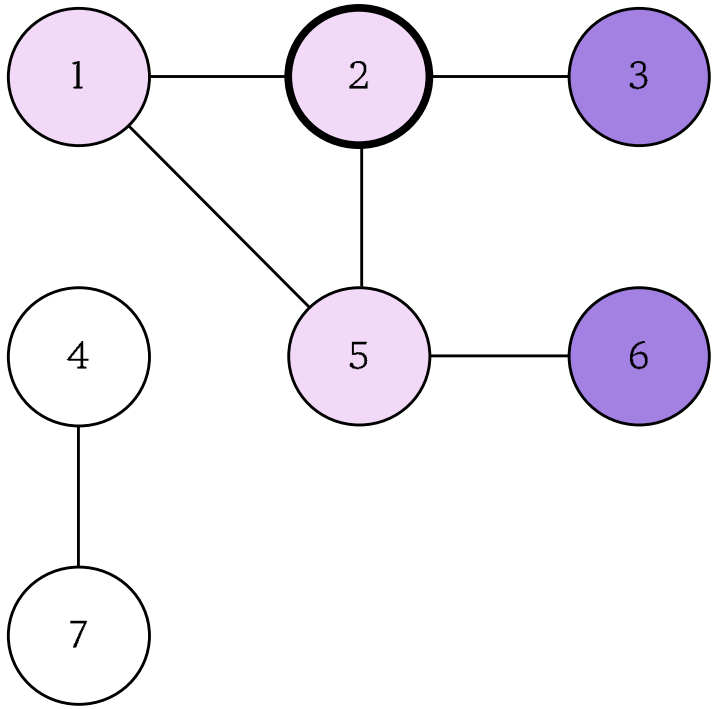
방문한 순서: 1, 5, 6, 2, 3
방문완료한 순서: 6

#DFS

<Depth First Search>

2번 정점도 더 이상 방문 가능한 정점이 없기에 방문을 완료하게 됩니다.

5번, 1번 정점도 마찬가지의 과정으로 방문을 완료하게 됩니다.



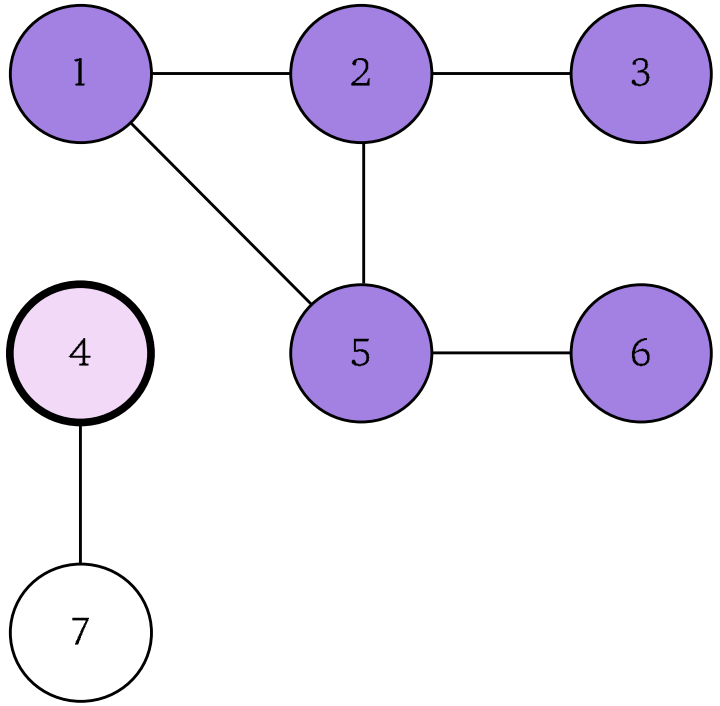
방문한 순서: 1, 5, 6, 2, 3
방문완료한 순서: 6, 3

#DFS

<Depth First Search>

현재 방문하지 않은 점에 대하여 위의 과정을 다시 반복하게 됩니다.

4번 정점에서 반복을 합니다.

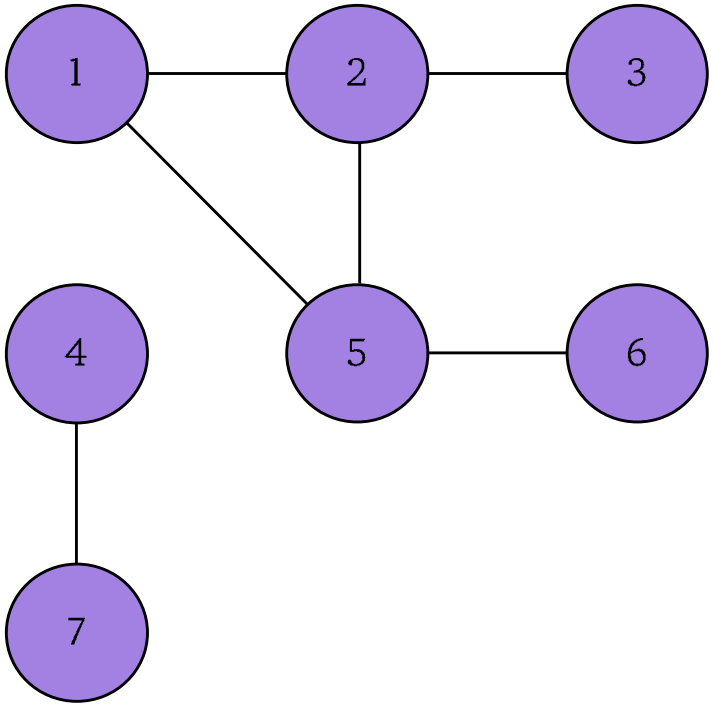


방문한 순서: 1, 5, 6, 2, 3, 4
방문완료한 순서: 6, 3, 2, 5, 1

#DFS

<Depth First Search>

모든 정점을 방문하게 되면 DFS를 종료하게 됩니다.



방문한 순서: 1, 5, 6, 2, 3, 4, 7
방문완료한 순서: 6, 3, 2, 5, 1, 7, 4

#DFS

<Depth First Search>

이제 위의 과정을 코드로 구현하면 다음과 같습니다.

Visited라는 방문여부를 저장하는 배열을 추가적으로 선언하게 됩니다.

DFS는 재귀적으로 구성하게 됩니다.
현재 정점에 대하여 방문표시를 한 후,
갈 수 있는 방문하지 않은 모든 정점에 대하여 반복합니다.

```
int ans = 0;
vector<int> g[111];
vector<bool> visited(111, false);


void dfs(int cur){
    visited[cur] = true;
    ans++;
    for(int i = 0; i < g[cur].size(); i++){
        int ch = g[cur][i];
        if(!visited[ch])
            dfs(ch);
    }
}

int main(){
    …
    dfs(1);
    …
}
```

#DFS

<Depth First Search>

그래프 외에서도 DFS를 활용할 수 있습니다.

옆은 배열에서 DFS를 사용하는 예시입니다.

일반적으로 DFS는

1. 현재 상태를 처리했다고 표시
2. 될 수 있는 다음 상태 중에서 처리하지 않은 것에 대하여 반복

의 구조를 가집니다.

```
int b[66][66];
bool visited[66][66];
int n;

void dfs(int x, int y){
    visited[x][y] = true;
    int d = b[x][y];
    if(x + d < n && !visited[x + d][y])
        dfs(x + d, y);
    if(y + d < n && !visited[x][y + d])
        dfs(x, y + d);
}
```

#DFS

<Depth First Search>



바이러스 (JUNGOL #1818)

1번 정점(컴퓨터)에서 시작하여 방문할 수 있는 정점들만 감염된다는 사실을 관찰할 수 있습니다.

따라서 1번 정점에서 DFS를 시행하여 방문할 수 있는 정점의 개수를 세면 됩니다.

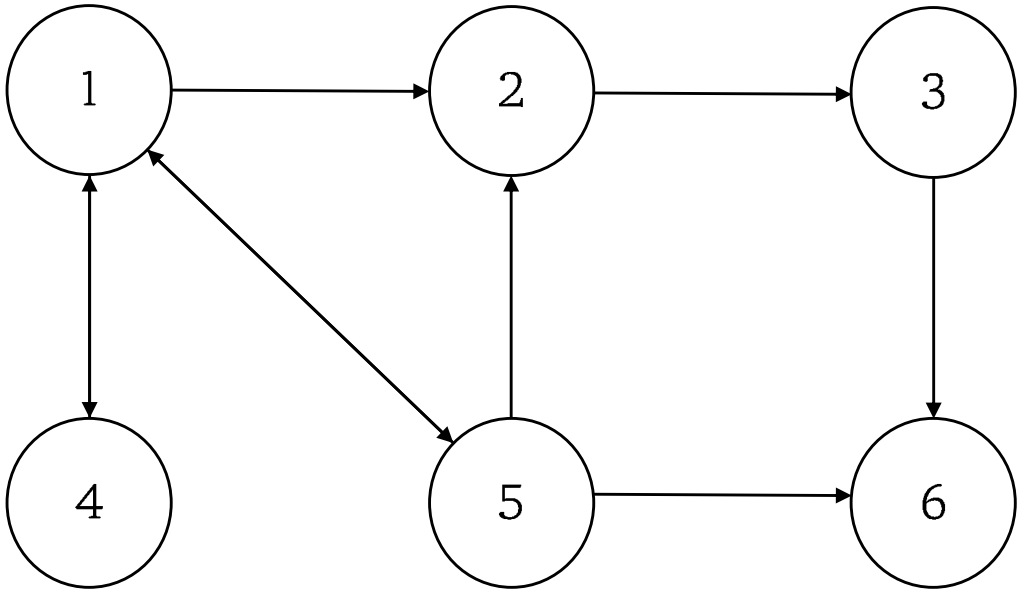
#BFS

<Breadth First Search>

DFS:Depth First Search(깊이 우선 탐색)

그래프를 순회하는 방법 중 하나입니다.

현재 탐색하는 경로를 끝까지 탐색하는 것을
반복하는 방식으로 그래프에 있는
모든 정점을 순회합니다.



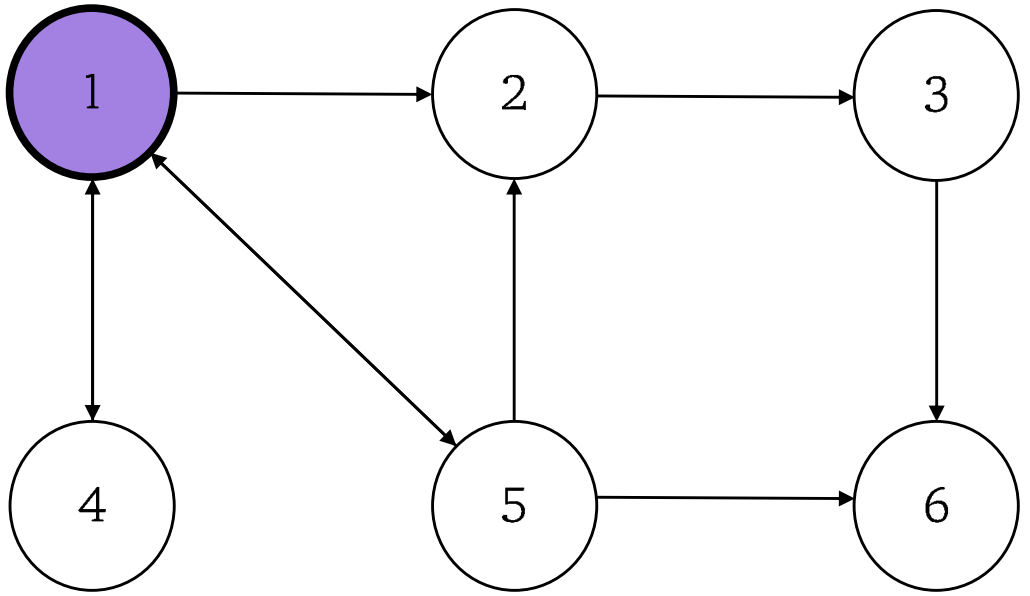
#BFS

<Breadth First Search>

DFS:Depth First Search(깊이 우선 탐색)

시작 정점을 1로 잡고 갈 수 있는 정점 중 최솟값을 우선으로 하여 DFS를 진행해봅시다.

그러면 DFS를 했을 때
1, 2, 3, 6, 4, 5의 순서로 방문하게 됩니다.



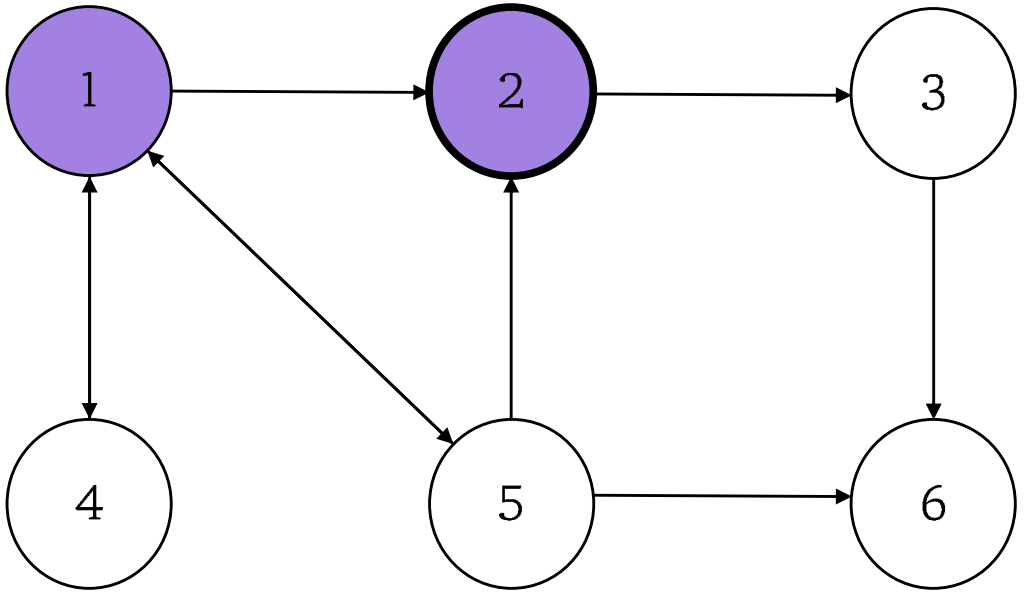
#BFS

<Breadth First Search>

DFS:Depth First Search(깊이 우선 탐색)

시작 정점을 1로 잡고 갈 수 있는 정점 중 최솟값을 우선으로 하여 DFS를 진행해봅시다.

그러면 DFS를 했을 때
1, 2, 3, 6, 4, 5의 순서로 방문하게 됩니다.



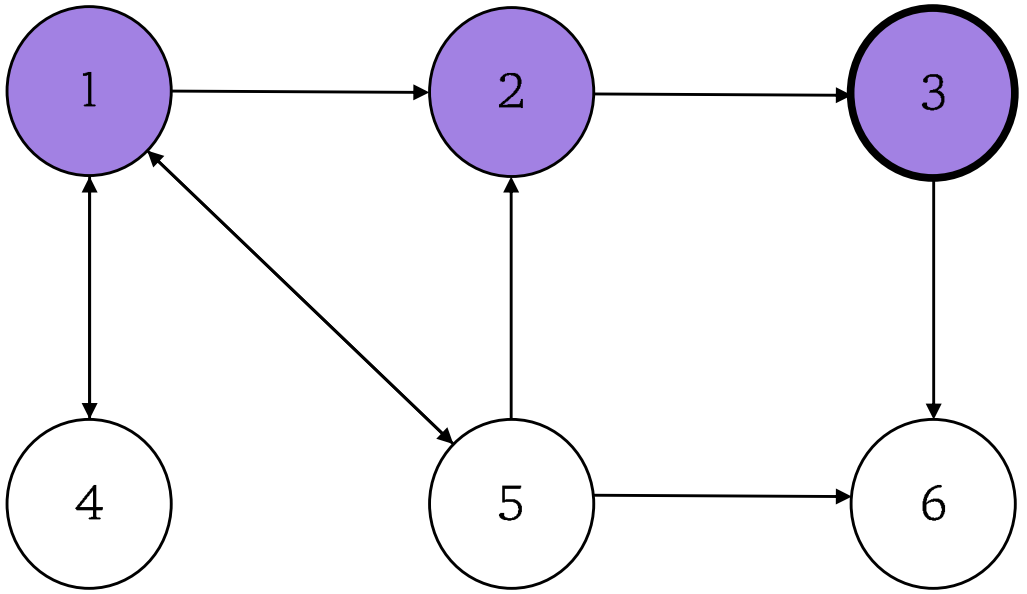
#BFS

<Breadth First Search>

DFS:Depth First Search(깊이 우선 탐색)

시작 정점을 1로 잡고 갈 수 있는 정점 중 최솟값을 우선으로 하여 DFS를 진행해봅시다.

그러면 DFS를 했을 때
1, 2, 3, 6, 4, 5의 순서로 방문하게 됩니다.



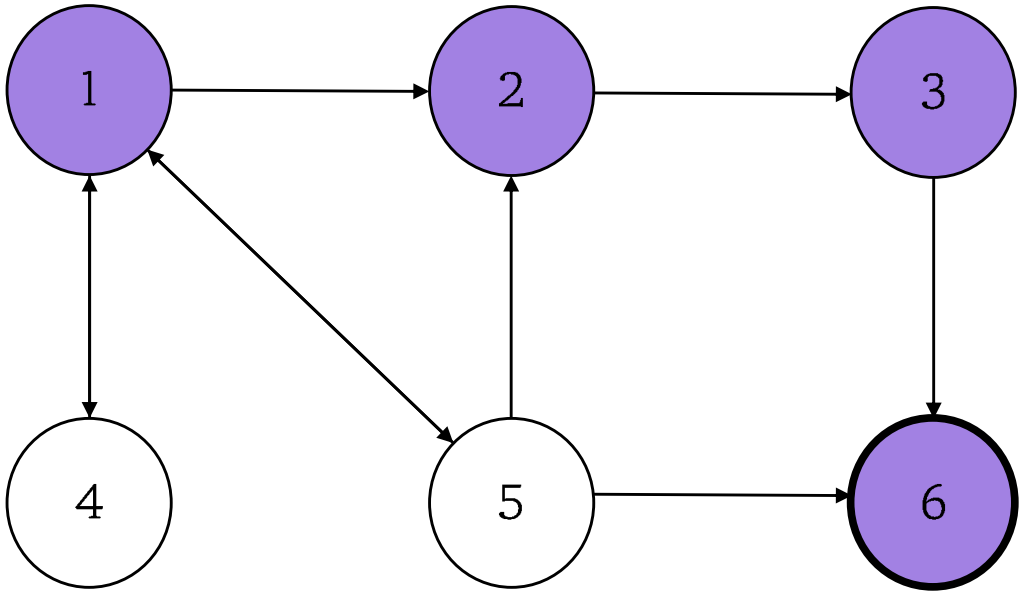
#BFS

<Breadth First Search>

DFS:Depth First Search(깊이 우선 탐색)

시작 정점을 1로 잡고 갈 수 있는 정점 중 최솟값을 우선으로 하여 DFS를 진행해봅시다.

그러면 DFS를 했을 때
1, 2, 3, 6, 4, 5의 순서로 방문하게 됩니다.



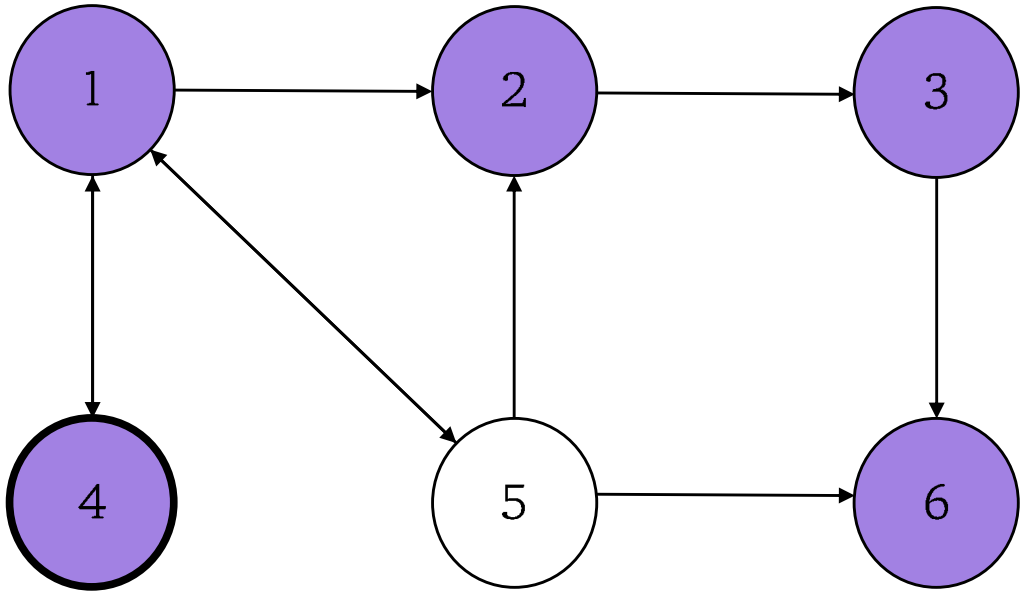
#BFS

<Breadth First Search>

DFS:Depth First Search(깊이 우선 탐색)

시작 정점을 1로 잡고 갈 수 있는 정점 중 최솟값을 우선으로 하여 DFS를 진행해봅시다.

그러면 DFS를 했을 때
1, 2, 3, 6, 4, 5의 순서로 방문하게 됩니다.



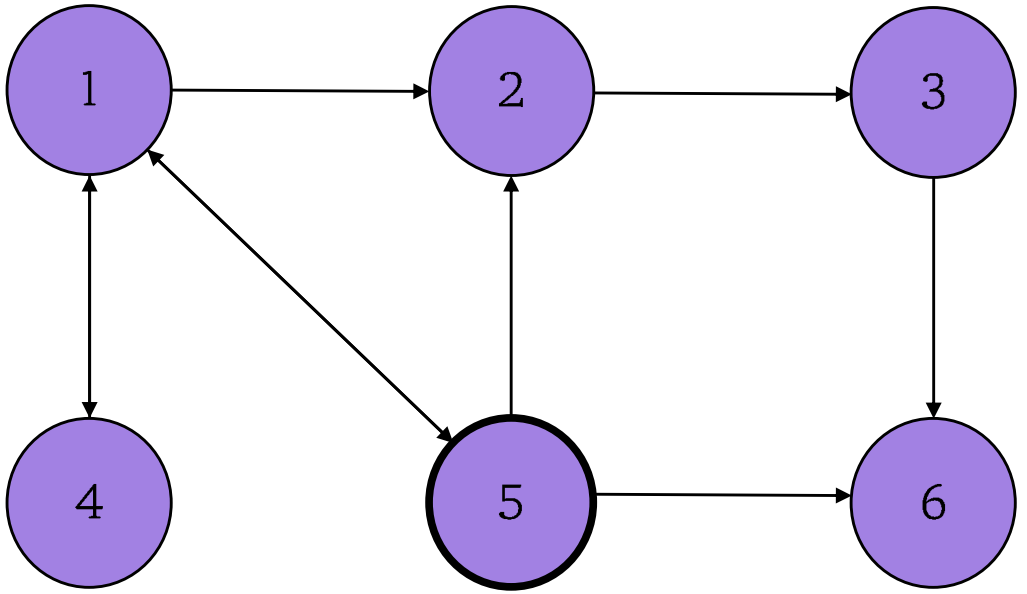
#BFS

<Breadth First Search>

DFS:Depth First Search(깊이 우선 탐색)

시작 정점을 1로 잡고 갈 수 있는 정점 중 최솟값을 우선으로 하여 DFS를 진행해봅시다.

그러면 DFS를 했을 때
1, 2, 3, 6, 4, 5의 순서로 방문하게 됩니다.



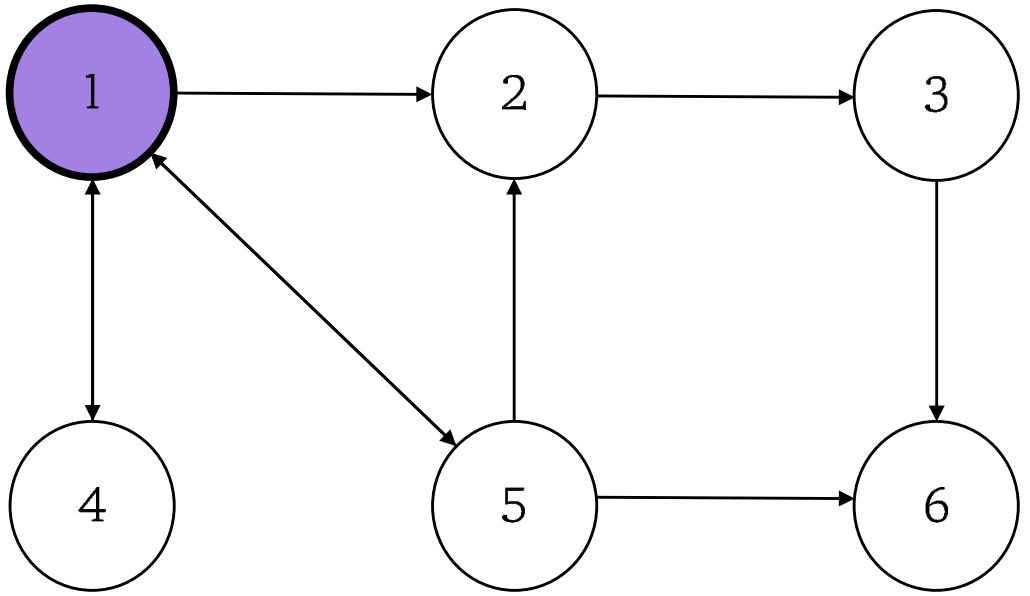
#BFS

<Breadth First Search>

BFS:Breadth First Search(너비 우선 탐색)

그래프를 순회하는 방법 중 하나입니다.

현재 정점에서 가장 인접한 정점을
우선으로 탐색하는 방법입니다.



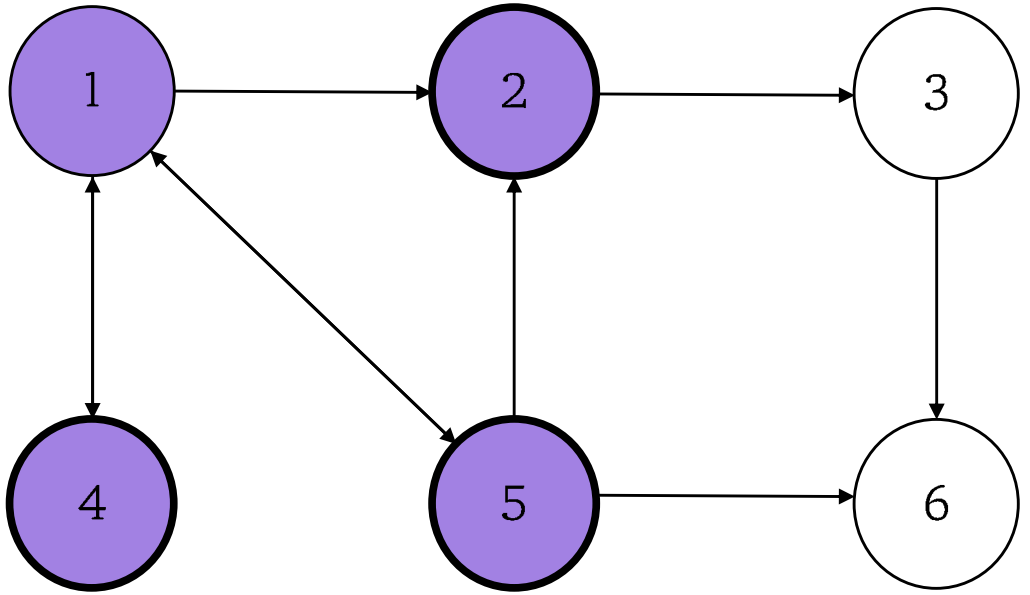
#BFS

<Breadth First Search>

BFS:Breadth First Search(너비 우선 탐색)

마찬가지로 1번 정점에서 시작하고 작은 번호의 정점을 우선으로 탐색하겠습니다.

BFS를 활용할 경우 탐색 순서는
1, 2, 4, 5, 3, 6이 됩니다.



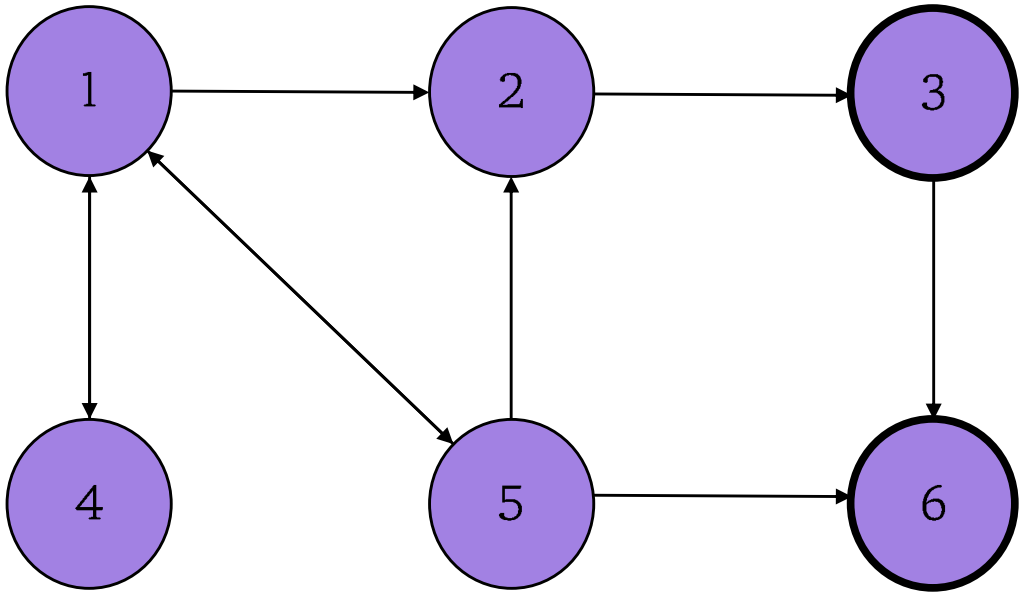
#BFS

<Breadth First Search>

BFS:Breadth First Search(너비 우선 탐색)

마찬가지로 1번 정점에서 시작하고 작은 번호의 정점을 우선으로 탐색하겠습니다.

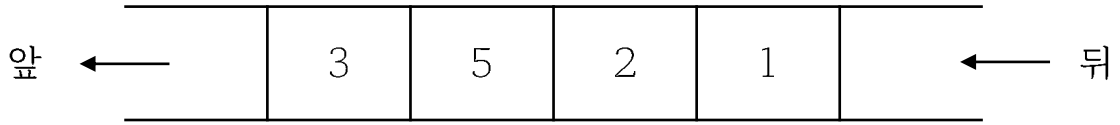
BFS를 활용할 경우 탐색 순서는
1, 2, 4, 5, 3, 6이 됩니다.



#BFS
<Breadth First Search>

BFS:Breadth First Search(너비 우선 탐색)

BFS를 구현할 때 저희는 Queue라는 자료구조를 사용하게 됩니다.



FIFO(First In First Out)의 형태로 먼저 들어온 자료가 먼저 나온다는 특징을 가집니다.

Queue는 앞, 뒤가 있는데 각각
앞: 조회, 제거
뒤: 추가
를 할 수 있습니다.

#BFS

<Breadth First Search>

BFS를 코드로 구현하면 다음과 같습니다.

DFS와 다르게 비재귀적으로 구현하게 됩니다.

마찬가지로 방문 여부를 저장하는 visited 배열을 선언한 후, queue에 시작 정점을 넣게 됩니다. 이후 queue가 빌 때까지 반복하여 BFS를 진행하게 됩니다.

방문 처리는 queue에 정점을 넣고 바로 해야 하는 것에 유의하세요!

```
vector<int> g[1234];
vector<bool> visited(1234,false);


void bfs(int start){
    queue<int> q;
    q.push(start);
    visited[start]=true;
    while(!q.empty()){
        int cur=q.front();
        q.pop();
        cout << cur <<" ";
        for(int next:g[cur]){
            if(visited[next])
                continue;
            q.push(next);
            visited[next] = true;
        }
    }
}
```

#BFS

<Breadth First Search>

Queue가 빌 때까지 다음을 반복하게 됩니다.

1. queue의 맨 앞 값을 cur에 저장 후 cur을 출력합니다.
2. queue의 맨 앞 값을 제거합니다.
3. cur에서 갈 수 있는 정점 중 아직 방문하지 않은 정점을 queue에 추가합니다.
4. 1-3의 과정을 반복합니다.

```
vector<int> g[1234];
vector<bool> visited(1234,false);


void bfs(int start){
    queue<int> q;
    q.push(start);
    visited[start]=true;
    while(!q.empty()){
        int cur=q.front();
        cout << cur <<" ";
        q.pop();
        for(int next:g[cur]){
            if(visited[next])
                continue;
            q.push(next);
            visited[next] = true;
        }
    }
}
```

#BFS

<Breadth First Search>

DFS와 BFS

DFS

1. 메모리 효율성이 높다.
2. 구현이 비교적 쉽다.
3. 찾는 답의 깊이가 깊은 경우 더 빠르다.

BFS

1. 답이 되는 경로가 여러 개인 경우 최단 경로임을 보장
2. 최단 경로가 존재할 때 반드시 찾을 수 있음
3. 찾는 답의 깊이가 얕은 경우 더 빠르다.

#BFS

<Breadth First Search>



단지번호붙이기 (JUNGOL #1695)

<문제 설명>

- 1과 0으로 이루어진 N×N 지도가 주어진다.
  - 1은 집이 있는 곳, 0은 집이 없는 곳이다.
- 상하좌우로 인접한 집을 단지라고 한다.
- 단지의 수와 각 단지에 속한 집을 출력하여라.

<제약 조건>

- $5 \leq N \leq 25$

#BFS

<Breadth First Search>



단지번호붙이기 (JUNGOL #1695)

격자 그래프에서 BFS를 사용하는 전형적인 문제입니다.
배열의 모든 값을 BFS를 활용하여 탐색하면 됩니다.

탐색을 할 때는 한 칸에서 상하좌우로 이동할 수 있습니다. 이를 그래프의 간선이라고 생각하면 됩니다.

0인 곳은 집이 없으니 탐색하면 안 되는 것에 주의합시다.

#BFS

<Breadth First Search>



단지번호붙이기 (JUNGOL #1695)

```cpp
int graph[105][105], visited[105][105]
    , dx[4] = {-1, 0, 0, 1}, dy[4] = {0, -1, 1, 0}, w, cnt;
vector<int> danji;
void bfs(int i, int j) {
    queue<pair<int, int>> q;
    q.push({i, j});
    visited[i][j] = 1;
    while (!q.empty()) {
        pair<int, int> cur = q.front();
        cnt++;
        q.pop();
        for (int i = 0; i < 4; i++) {
            int nx = cur.first + dx[i], ny = cur.second + dy[i];
            if (nx >= 0 && nx < w && ny >= 0 && ny < w) {
                if (visited[nx][ny] || graph[nx][ny] == 0) continue;
                q.push({nx, ny});
                visited[nx][ny] = 1;
            }
        }
    }
}
```

```cpp
int main() {
    cin.tie(0)->sync_with_stdio(0);
    cin >> w;
    for (int i = 0; i < w; i++) {
        string s; cin >> s;
        for (int j = 0; j < w; j++) {
            graph[i][j] = s[j] - '0';
        }
    }
    for (int i = 0; i < w; i++) {
        for (int j = 0; j < w; j++) {
            if (visited[i][j] || graph[i][j] == 0) continue;
            cnt = 0;
            bfs(i, j);
            danji.push_back(cnt);
        }
    }
    sort(danji.begin(), danji.end());
    cout << danji.size() << "\n";
    for (int i = 0; i < danji.size(); i++)
        cout << danji[i] << "\n";
    return 0;
}
```

#BFS

<Breadth First Search>



단지번호붙이기 (JUNGOL #1695)

```
dx[4] = {-1, 0, 0, 1}, dy[4] = {0, -1, 1, 0}
```

dx와 dy는 격자에서 상하좌우로 움직이는 값을 저장합니다.

```
for (int i = 0; i < 4; i++) {
    int nx = cur.first + dx[i], ny = cur.second + dy[i];
        if (nx >= 0 && nx < w && ny >= 0 && ny < w) {
            if (visited[nx][ny] || graph[nx][ny] == 0) continue;
            q.push({nx, ny});
            visited[nx][ny] = 1;
        }
    }
}
```

nx와 ny는 상하좌우로 움직인 이후의 좌표를 저장합니다. nx와 ny가 지도를 벗어나는지 검사해야 합니다.

#연습 문제



바이러스 (BOJ #1818)

위에서 설명한 문제입니다.



단지번호붙이기 (JUNGOL #1695)

위에서 설명한 문제입니다.



화염에서탈출 (JUNGOL #1082)

BFS를 할 때 queue에 시작 지점을 여러 개 넣을 수 있습니다.



토마토(고) (JUNGOL #2613)

day[i][j]를 해당 위치의 토마토가 익게 되는 최소의 날로 정의하고 BFS를 돌립시다.

#연습 문제



영역 구하기 (JUNGOL #1457)

단지번호붙이기와 비슷한 문제.



종교 (JUNGOL #1863)

연결된 그룹의 개수는?



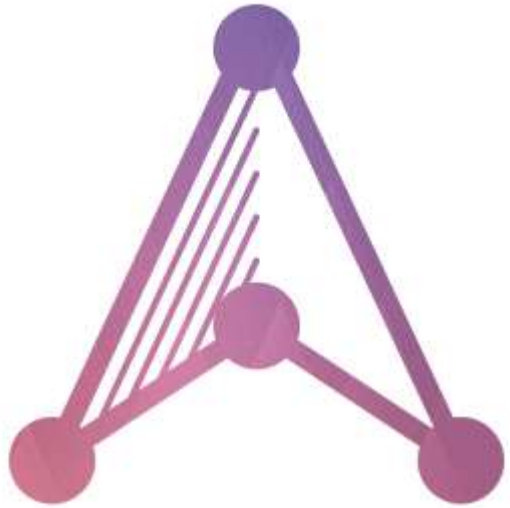
토마토(초) (JUNGOL #2606)

3차원이 추가되었어요.



경로 찾기 (JUNGOL #2261)

BFS로 최단 경로를 찾고 역추적하기



ALOHA

The algorithm club.